In [1]:
```
# Load Data
import pandas as pd

df = pd.read_csv("HousePricePrediction.csv")
```

In [2]:
```
df.shape
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2919 entries, 0 to 2918
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   Id            2919 non-null   int64  
 1   MSSubClass    2919 non-null   int64  
 2   MSZoning      2915 non-null   object 
 3   LotArea       2919 non-null   int64  
 4   LotConfig     2919 non-null   object 
 5   BldgType      2919 non-null   object 
 6   OverallCond   2919 non-null   int64  
 7   YearBuilt     2919 non-null   int64  
 8   YearRemodAdd  2919 non-null   int64  
 9   Exterior1st   2918 non-null   object 
 10  BsmtFinSF2    2918 non-null   float64
 11  TotalBsmtSF   2918 non-null   float64
 12  SalePrice     1460 non-null   float64
dtypes: float64(3), int64(6), object(4)
memory usage: 296.6+ KB
```

Out[2]:

	Id	MSSubClass	LotArea	OverallCond	YearBuilt	YearRemodA
count	2919.000000	2919.000000	2919.000000	2919.000000	2919.000000	2919.000(
mean	1459.000000	57.137718	10168.114080	5.564577	1971.312778	1984.264
std	842.787043	42.517628	7886.996359	1.113131	30.291442	20.894
min	0.000000	20.000000	1300.000000	1.000000	1872.000000	1950.000(
25%	729.500000	20.000000	7478.000000	5.000000	1953.500000	1965.000(
50%	1459.000000	50.000000	9453.000000	5.000000	1973.000000	1993.000(
75%	2188.500000	70.000000	11570.000000	6.000000	2001.000000	2004.000(
max	2918.000000	190.000000	215245.000000	9.000000	2010.000000	2010.000(

In [3]:
```
# Data Preprocessing - Data cleaning

# Drop Id as it doesn't contribute in price
df.drop(['Id'],
             axis=1,
             inplace=True)

# Replacing SalePrice empty values with their mean values
df['SalePrice'] = df['SalePrice'].fillna(
  df['SalePrice'].mean())
```

```
# Drop records with null values
df = df.dropna()

df.isnull().sum()
df.head()
```

Out[3]:

	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt	Year
0	60	RL	8450	Inside	1Fam	5	2003	
1	20	RL	9600	FR2	1Fam	8	1976	
2	60	RL	11250	Inside	1Fam	5	2001	
3	70	RL	9550	Corner	1Fam	5	1915	
4	60	RL	14260	FR2	1Fam	5	2000	

In [4]:
```
# Data Preprocessing - Encoding (Covered in Supervised ML Part2)
cols = ['MSZoning', 'LotConfig', 'BldgType', 'Exterior1st']
df = pd.get_dummies(df, columns=cols, drop_first=True)
```

In [5]:
```
# Train-Test Split
from sklearn.model_selection import train_test_split

X = df.drop(['SalePrice'], axis=1)
Y = df['SalePrice']

X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.2, random_state=0)
```

In [6]:
```
X_train.head()
```

Out[6]:

	MSSubClass	LotArea	OverallCond	YearBuilt	YearRemodAdd	BsmtFinSF2	Totall
1453	20	17217	5	2006	2006	0.0	
2021	20	19508	5	1974	1974	0.0	
188	90	7018	5	1979	1979	0.0	
1273	80	11512	7	1959	2006	0.0	
1960	20	7560	5	1971	1971	613.0	

5 rows × 33 columns

In [7]:
```
# Train Linear Regression Model
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

Out[7]:

▾ LinearRegression

▶ Parameters

In [8]:
```
# Model Evaluation
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.metrics import mean_absolute_percentage_error
import numpy as np

y_pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print(mean_absolute_percentage_error(y_test, y_pred))
```

```
R2 Score: 0.37414217961383156
MAE: 30829.93666432242
RMSE: 41138.558702301765
0.18742025463428386
```

In [9]:
```
# Feature Scaling - to try & improve baseline performance
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LinearRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print(mean_absolute_percentage_error(y_test, y_pred))

# To get better results we can try some other regression model
# & also use techniques like Bagging & Boosting (will cover later).
```

```
R2 Score: 0.3741421796138532
MAE: 30829.93666432187
RMSE: 41138.55870230105
0.18742025463427797
```